



Agenda

Longest Substring Without Repeat

First Non Repeating Element

Check if there exists a subarray with sum = 0

Check if there exists a subarray with sum = K

Given a string s , find the length of the longest substring without repeating characters.

"a b c a b c b b"

substring $\rightarrow$ continuous part
of a string.

ans = 3 //

"b b b b b"

"p w w k e w"

$\rightarrow$ 3 //

ans = 1

Brute force: Generate all substring $\rightarrow O(N^2)$
 $\hookrightarrow$ for each substring check for all unique characters. $\rightarrow O(N)$

9:18 $\rightarrow$ 9:26.

$T(C: O(N^3))$ $SC: O(1)$

$[a, b] = \underline{b - a + 1}$

hint $\rightarrow$ sliding window

"a b c a b c b b"

3 e $\rightarrow$ continue

max = ~~4~~ $\times$ ~~3~~ 3

ans = 1, 2, 3, 3, 3, 3, 2

$\rightarrow$ b, x, ✓

	c	b
✓	x	
x	b	
x		

Code $\rightarrow$ 10 min

9:38 $\rightarrow$ 9:48

function lengthOfLongestSubstring(s):

initialize start = 0

initialize maxLength = 0

initialize hashSet = empty set

for end = 0 to length of s - 1:

while s[end] in hashSet:

remove s[start] from hashSet

increment start by 1

add s[end] to hashSet

maxLength = max(maxLength, end - start + 1)

return maxLength

Given N elements, find the first non-repeating element.

N=6

A = { 1, 2, 3, 1, 2, 5 }

↓
first non repeating
element.

5 min

9:55 pm

↓

10:00 pm

A = { 4, 3, 3, 2, 5, 6, 4, 5 }

↓ ↓ ↓ ↓
2 2 2 1
first non-repeating.

Idea: 1 1) Create a hashmap, store <element, frequency>

4	→ 2
3	→ 2
2	→ 1
5	→ 1
6	→ 1

→ ✗ 2) traverse on hashmap and check for element with freq == 1.
Because hashmap/map → doesn't store elements in order.

2) → traverse on Array → whichever element first has freq == 1
↓
that's our ans.

pseudocode

Function firstNonRepeating(A[])

{
 HashMap<integer, integer> mp;
 n = A.length

direct

→ creating hashmap / map.

10 min

```
for(i -> 0 to n - 1)
{
    if(mp.Search(A[i]) == true)
    {
        mp[A[i]] ++
    }
    else{
        mp.Insert(A[i], 1)
    }
}
```

→ inserting and updating h.m

10:06

↓

10:16pm

mp[A[i]] > 1

```
for(i -> 0 to n - 1)
{
    if(mp[A[i]] == 1)
    {
        return A[i]
    }
}
```

→ checking for first unique element.

return - 1

}

→ Given an array of N elements, check if there exists a subarray with a sum equal to 0.

$A = \{ 2, 2, 2, -3, 4, 3, 1, -2, -3, 2 \}$

$$\downarrow$$

$$2 + 1 - 3 = 0 //$$

Idea →

Bruteforce → $O(N^3)$

Prefix sum, convert to $O(N^2)$ → $O(N)$

↓
check??
 $O(N)$

Can we use hashmap / hashset

→ × $A = \{ 2, 2, 2, -3, 4, 3, 1, -2, -3, 2 \}$

→ ✓ $Pf = \{ 2, 4, 5, 2, 6, 9, 10, 8, 5, 7 \}$

10: 25pm

↓

10: 35pm

↓

$$\text{sum}[i \text{ to } j] == 0.$$

$\begin{matrix} 5, 6 \\ 2, 2 \end{matrix}$

$$Pf[i, j] = Pf[j] - Pf[i-1]$$

$$Pf[j] - Pf[i-1] == 0$$

$$Pf[j] == Pf[i-1].$$

$A = \{ -2, -1, 3, 5 \}$

$Pf = \{ -2, -3, \underline{0}, 5 \}$

- 1) check for duplicate
 - 2) check for 'zero'
- } in prefix array

// 1. todo calculate prefix sum array $\rightarrow O(N)$

// 2.

Function checkSubArraySumZero(PrefixSumArray[])

```
{  
    HashSet<integer> s  
    for(i -> 0 to PrefixSumArray.length - 1 )  $\rightarrow O(N)$   
    {  
        if(PrefixSumArray[i] == 0)  
        {  
            return true  
        }  
        s.insert(PrefixSumArray[i])  
    }  
    if (s.size != PrefixSumArray.size)  
        return true  
    return false  
}
```

$2N \rightarrow$

TC : $O(N)$
SC : $O(N)$

Given an array $arr[n]$ check if there exists a subarray with sum = K

$A = \overset{0}{1} \overset{1}{2}, \overset{2}{3}, \overset{3}{9}, \overset{4}{-4}, \overset{5}{1}, \overset{6}{5}, \overset{7}{6}, \overset{8}{2}, \overset{9}{5}$

$k = 11 \rightarrow \{5, 6, 3\}, \{2, 3, 9, -4, 1\}$

$k = 10 \rightarrow \{2, 3, 9, -4\}$

$k = 15 \rightarrow \{-4, 1, 5, 6, 2, 5\}$

$A = [5, \underline{10}, 20, 100, 105]$ $k = 110 \rightarrow \times$

$k = 10 \checkmark$

$$\text{pf}[j] - \text{pf}[i-1] = k$$
$$a - b = k - a$$

hashset ??

TC: $O(N)$

SC: $O(1)$ $\rightarrow ? \rightarrow \times$

hashset

pfsum array $\{ \rightarrow ??$

$\downarrow$
carry forward

$$k = 11 \rightarrow \text{subarray sum} \rightarrow a - b = k - a \quad \text{---} \quad -b = k - a \quad \boxed{b = a - k}$$

$$P[j] - P[i-1] = k \quad k = 11 \quad 2$$

$A = \{ 2, 3, 9, -4, 1, 5, 6, 2, 5 \}$

Pf array $\rightarrow$ Cum sum $\rightarrow$ 2, 5, 14, 10, 11, 16

Target $\downarrow \downarrow \downarrow \downarrow \downarrow$

a-k -9 -6 3 -1 0 5

Subarray exist

Hashset 2
 2, 5, 14
 10, 11

Edge case

$A = \{ 2, 3, 9, -4, 1 \}$

Pf sum $\rightarrow$ Cum sum $\rightarrow$ 2, 5, 14, 10

target $\downarrow \downarrow \downarrow \downarrow$

a-k -8 -5 4 0 1
 x x x x x

a=10, b = a - k
b = 10 - 10 = 0

hashset.

2, 5, 14, 10
1.

Solution

insert 0 initially

hashset.

Insert $\rightarrow$ 0, 2, 5, 14, 10
1.

Note: for above edge case add '0' to hashset before traversing.

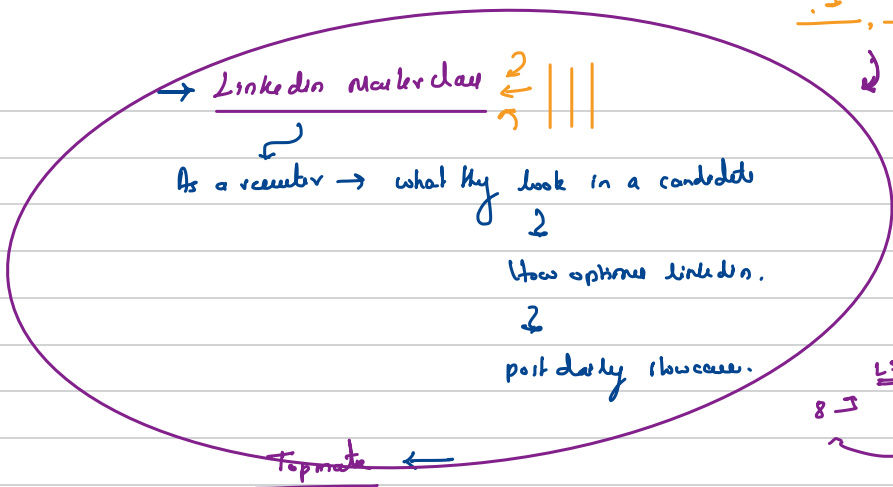
Pseudocode :-

TC: $O(N)$
SC: $O(N)$

```
function targetSubarray(arr[], K){
    N = arr.length;
    a = 0;

    //cumulative sum, long -> to avoid overflow
    HashSet<long integer> hs;
    hs.add(0);
    for(i -> 0 to N - 1){
        a = a + arr[i];

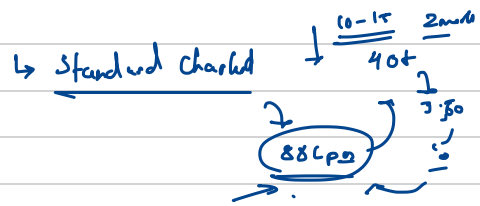
        //cumulative sum = target = a - k
        if(hs.contains(a - K)){ →
            //subarray exists
            return true; → ? {
        } else {
            hs.add(a);
        }
    }
    return false;
}
```



Good time

No time, lean to his, time, Good time

time, time, time



advanced BSA (scalar)

leetcode contest

tabular hell

gave best indicator

↓

revision class (multiple)

↓

draft resolution

↓

Influence!

lot of contest

→ give contest

Influence? ✓

lack of content

Leetcode



Assignment + additional → search for it in leetcode

2

solve

leetcode, codechef → contest.

1 leetcode

roadmap.sh